



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Quizzie - Plataforma multidispositivo para la realización y gestión dinámica de tests educativos

Realizado por
Francisco Gago Vázquez

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Jesús Moreno León

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de julio, curso 2025/26

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Citamos para temporalmente para evitar errores en la parte de bibliografía [[1](#)]

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: Keyword 1, keyword 2, ..., keyword N

Índice general

1	Introducción	1
1.1.	Contexto y objeto del proyecto	1
1.2.	Motivación	2
1.3.	Objetivos del proyecto	2
1.3.1.	Objetivos académicos	2
1.3.2.	Objetivos técnicos y profesionales	3
1.4.	Resumen del <i>stack</i> tecnológico	3
1.4.1.	Ecosistema del <i>backend</i>	4
1.4.2.	Ecosistema del <i>frontend</i>	4
1.5.	Estructura de la memoria	4
2	Acta de constitución	7
2.1.	Información del proyecto	7
2.2.	Descripción y propósito	7
2.3.	Requerimientos de alto nivel	8
2.4.	Marco legal y normativo	9
2.5.	Riesgos iniciales de alto nivel	9
2.6.	Lista de interesados (<i>Stakeholders</i>)	10
3	Gestión del proyecto	12
3.1.	Metodología del trabajo	12
3.2.	Línea base del alcance	13
3.2.1.	Estructura del Desglose del Trabajo (EDT)	13
3.2.2.	Diccionario de la EDT	14
3.3.	Línea Base del Cronograma	21
3.3.1.	Lista de actividades	21
3.3.2.	Lista de hitos	21
3.3.3.	Diagrama de Gantt	21
3.4.	Línea Base de Costes	21
3.4.1.	Análisis de costes	22
3.4.2.	Costes de infraestructura según el número de usuarios	22
3.4.3.	Coste total de propiedad (TCO)	22
3.4.4.	Plan de gestión de cambios y riesgos	22
4	Estado del arte y fundamentación	23
4.1.	Análisis del mercado	23
4.2.	Factores diferenciadores	23
4.3.	Fundamentación tecnológica	23
4.3.1.	Limitaciones tecnológicas	23
4.3.2.	Alternativas consideradas	23
4.3.3.	Justificación de la decisión	23
4.4.	Conclusiones del estudio previo	23

5	Análisis de requisitos	24
5.1.	Actores del sistema	24
5.2.	Historias de usuario	24
5.3.	Requisitos de información	24
5.4.	Reglas de negocio y restricciones	24
5.5.	Requisitos funcionales	24
5.5.1.	Criterios de aceptación	24
5.6.	Casos de uso	24
5.7.	Requisitos no funcionales	24
5.8.	Trazabilidad de requisitos	25
6	Diseño y arquitectura	26
6.1.	Patrón arquitectónico elegido	26
6.2.	Estructura modular del código	26
6.3.	Modelo de datos	26
6.4.	Diseño de interfaces	26
6.5.	Seguridad	26
6.6.	Documentación de la API	27
7	Metodología técnica (CI/CD)	28
7.1.	Políticas del repositorio	28
7.2.	Integración y Despliegue Continuo (CI/CD)	28
7.2.1.	Configuración del Pipeline y automatización	28
7.3.	Perfiles de configuración y entornos	28
7.4.	Métricas de calidad y análisis estático de código	28
8	Implementación y pruebas	30
8.1.	Componentes clave	30
8.2.	Estrategia de testing	30
8.3.	Casos de prueba y validación	30
8.4.	Pruebas de carga y rendimiento	30
9	Resultados y evaluación	31
9.1.	Grado de cumplimiento de la Línea Base del Alcance	31
9.2.	Tiempo real frente a Línea Base del Cronograma	31
9.3.	Gastos incurridos frente a Línea Base de Costes	31
9.4.	Desviaciones y gestión de cambios realizados	31
9.5.	Resultados de las pruebas y validación	31
9.6.	Retrospectiva técnica	31
10	Conclusiones	32
10.1.	Grado de cumplimiento de los objetivos	32
10.2.	Lecciones aprendidas y valoración personal	32
10.3.	Trabajos futuros y líneas de evolución	32
	Bibliografía	35

Índice de figuras

3.1. Estructura de Desglose del Trabajo (EDT).	14
--	----

Índice de tablas

2.1. Datos de identificación del proyecto.	7
2.2. Riesgos iniciales y estrategias de mitigación.	10
2.3. Lista de interesados y roles en el proyecto.	11
3.1. Datos del entregable 1.1.	14
3.2. Actividades del entregable 1.1.	14
3.3. Datos del entregable 1.2.	15
3.4. Actividades del entregable 1.2.	15
3.5. Datos del entregable 1.3.	15
3.6. Actividades del entregable 1.3.	16
3.7. Datos del entregable 2.1.	16
3.8. Actividades del entregable 2.1.	16
3.9. Datos del entregable 2.2.	16
3.10. Actividades del entregable 2.2.	17
3.11. Datos del entregable 3.1.	18
3.12. Actividades del entregable 3.1.	18
3.13. Datos del entregable 3.2.	18
3.14. Actividades del entregable 3.2.	19
3.15. Datos del entregable 3.3.	19
3.16. Actividades del entregable 3.3.	19
3.17. Datos del entregable 4.1.	20
3.18. Actividades del entregable 4.1.	20
3.19. Datos del entregable 4.2.	20
3.20. Actividades del entregable 4.2.	20
3.21. Datos del entregable 4.3.	21
3.22. Actividades del entregable 4.3.	21

Índice de extractos de código

1. Introducción

Es evidente que la **Inteligencia Artificial generativa** está cambiando nuestra forma de trabajar, aprender y comunicarnos. Su uso puede potenciar claramente nuestro potencial, pero también puede suponer un riesgo a nivel educativo, ya que el acceso inmediato a la información puede provocar una gran disminución de la motivación para el **estudio autónomo** y el **pensamiento crítico** en los estudiantes, quienes podrían ver inútil el esfuerzo personal.

Ante esta problemática, el presente proyecto presenta *Quizzie*, una plataforma diseñada para ofrecer una solución dinámica y sencilla que permite a los docentes validar los conocimientos adquiridos directamente en el aula. De este modo, se fomenta una **evaluación en tiempo real** que mitiga el abuso de la IA sin caer en metodologías tradicionales que aburran al estudiante.

1.1. Contexto y objeto del proyecto

A raíz del contexto tecnológico actual, surge la necesidad de buscar soluciones que adapten la forma de evaluar en las aulas a la realidad de los estudiantes. En la primera reunión con el tutor de este proyecto, planteamos el reto de diseñar una herramienta que facilitara el trabajo de los profesores y que, al mismo tiempo, resultara entretenida y motivadora para los alumnos a la hora de comprobar lo aprendido mediante cuestionarios en clase.

La integración de este tipo de cuestionarios rápidos ofrece grandes ventajas en el día a día del aula. Para el alumno, rompe la monotonía de las clases teóricas convencionales introduciendo **aprendizaje dinámico**, elevando la participación y proporcionando *feedback* instantáneo sobre su nivel de comprensión. Por otro lado, para el profesor funciona como una herramienta de **seguimiento en tiempo real**: le permite medir de manera automatizada si el grupo está asimilando los conceptos explicados y, en caso de detectar fallos comunes, aclarar las dudas en ese mismo momento.

Sin embargo, el uso de las plataformas actuales en entornos educativos reales destapa un **problema de control crítico**. Al basarse generalmente en la difusión de un enlace o un código de acceso, es muy sencillo que los estudiantes compartan estas credenciales por grupos de mensajería. Esto provoca que alumnos que no han asistido a clase, o que se encuentran en su casa, puedan responder al cuestionario a distancia. Esta falta de **verificación de la presencia física** falsea las estadísticas de rendimiento que recibe el docente e impide usar estos test como herramientas de **evaluación formal**.

Como respuesta a este escenario, el objeto de este proyecto es el desarrollo de *Quizzie*. La aplicación está diseñada para mantener la sencillez y el dinamismo que se busca en los cuestionarios en vivo, pero introduciendo una **capa de verificación**

para los alumnos. A través de este mecanismo de control, *Quizzie* asegura que la participación se acote estrictamente a los alumnos presentes en el aula, evitando el **fraude a distancia** y consolidándose como un instrumento de evaluación válido y seguro.

1.2. Motivación

La elección de este proyecto nace, en gran medida, de mi propia experiencia como estudiante a lo largo de mi formación. He vivido en primera persona cómo la introducción de **cuestionarios interactivos** en mitad de una clase es capaz de transformar por completo el clima del aula, despertando el interés de los alumnos y fijando los conceptos de una manera mucho más eficaz que los métodos tradicionales. No obstante, también he sido testigo de las limitaciones de las herramientas actuales: el uso de pseudónimos informales, la posibilidad de participar de forma remota sin acudir a clase o la frustración del docente al no poder **validar la autenticidad** de los resultados. Ver cómo una dinámica con tanto potencial perdía rigor por falta de control o por limitaciones de las plataformas utilizadas fue el detonante que me impulsó a buscar esta solución.

Desde un punto de vista profesional, mi motivación principal se centra en el reto de plasmar de forma integral todos los conocimientos y competencias adquiridos a lo largo de la carrera. El desarrollo de *Quizzie* representa la oportunidad perfecta para demostrar mi capacidad de afrontar un **proyecto de software completo**, responsabilizándome de cada una de sus fases: desde las etapas iniciales de concepción, estudio previo y análisis de requisitos, hasta el diseño arquitectónico, la implementación técnica y la validación mediante pruebas. Abordar este **ciclo de vida de extremo a extremo** no solo me permite consolidar mis conocimientos técnicos, sino también validar mi criterio al tomar decisiones justificadas ante problemas, convirtiendo este trabajo en el puente definitivo hacia mi madurez como ingeniero de *software*.

1.3. Objetivos del proyecto

Para guiar el desarrollo del proyecto y asegurar el cumplimiento de las expectativas tanto formativas como técnicas, he definido una serie de metas divididas en dos vertientes: académica y técnico-profesional.

1.3.1. Objetivos académicos

Metas orientadas a consolidar la formación recibida durante los estudios de grado:

- **Integrar y aplicar** de forma conjunta los conocimientos teóricos y prácticos adquiridos en las distintas asignaturas de la carrera.

- **Adoptar metodologías de trabajo ágiles** y estándares de la industria en la gestión y planificación temporal del proyecto.
- **Fomentar el uso de buenas prácticas** de ingeniería del *software*, incluyendo el control de versiones formal, el diseño de código limpio y la documentación sistemática.

1.3.2. Objetivos técnicos y profesionales

Retos tecnológicos asumidos para el producto final y el desarrollo de competencias laborales:

- **Dominar nuevas tecnologías** y herramientas del ecosistema de desarrollo actual, tales como el *framework* FastAPI para el entorno de servidor y React para la interfaz de usuario.
- **Gestionar de manera autónoma el ciclo de vida completo** de un producto de *software* real, abarcando desde su concepción y análisis hasta su despliegue y evaluación.
- **Diseñar e implementar un sistema de verificación** robusto y eficiente que resuelva de manera efectiva la problemática de la asistencia y autoría en las evaluaciones en el aula.
- **Asegurar la calidad del *software*** mediante el diseño y ejecución de un plan de pruebas automatizadas y la configuración de *pipelines* de integración continua.
- **Implementar una estrategia de pruebas exhaustiva** que abarque *tests* unitarios y de integración, asegurando la fiabilidad del código y el correcto funcionamiento de las reglas de negocio del sistema.
- **Diseñar y configurar un flujo de trabajo basado en CI/CD** (Integración y Despliegue Continuo) mediante herramientas de automatización, garantizando que cada cambio en el repositorio se verifique y despliegue de manera segura y eficiente.

1.4. Resumen del *stack* tecnológico

Para garantizar el cumplimiento de los objetivos técnicos del proyecto y asegurar un desarrollo robusto, eficiente y escalable, se ha seleccionado un conjunto de herramientas alineadas con los estándares de la industria actual. Esta sección ofrece una visión inicial del *stack* tecnológico empleado para la construcción de *Quizzie*. Las ventajas técnicas de cada opción frente a otras alternativas del mercado se argumentarán en profundidad en el Capítulo 4.

El núcleo del sistema se divide claramente bajo una arquitectura cliente-servidor, donde el *backend* gestiona la lógica de negocio, la seguridad y la comunicación en tiempo real, mientras que el *frontend* ofrece una experiencia interactiva y fluida para profesores y alumnos.

1.4.1. Ecosistema del *backend*

El entorno se ha diseñado priorizando el alto rendimiento, la seguridad y la agilidad en el desarrollo. Las tecnologías y herramientas principales que componen esta capa son:

- **Lenguaje de programación:** Python 3.12, aprovechando las últimas mejoras de rendimiento y tipado estático del lenguaje.
- **Framework de desarrollo:** FastAPI, seleccionado por su alta velocidad de ejecución, soporte nativo de operaciones asíncronas y validación automática de datos.
- **Comunicación en tiempo real:** Protocolo WebSocket, implementado para gestionar de manera bidireccional y eficiente el flujo de eventos y la sincronización de las salas en vivo.
- **Persistencia y ORM:** SQLAlchemy en combinación con SQLite. Al no requerir un gestor de base de datos externo complejo en esta fase, esta solución permite interactuar de forma nativa con la base de datos utilizando objetos de Python de manera eficiente.
- **Gestión de dependencias:** *uv*, utilizado como gestor de paquetes ultrarrápido para optimizar los tiempos de instalación y asegurar la reproducibilidad del entorno.

1.4.2. Ecosistema del *frontend*

La interfaz de usuario se ha planteado como una *Single Page Application* (SPA) responsiva para ofrecer la fluidez que requiere una aplicación con dinámicas lúdicas y en vivo:

- **Framework principal:** React, para la construcción de una interfaz basada en componentes reutilizables y reactivos.
- **Herramienta de construcción (Build Tool):** Vite, que proporciona un entorno de desarrollo ágil gracias a su velocidad de compilación y recarga en caliente.
- **Lenguajes:** TypeScript como eje principal para dotar al código de tipado fuerte y robustez, apoyado puntualmente en JavaScript.
- **Enrutado y consumo de API:** React Router Dom para la navegación interna de la aplicación y Axios para realizar peticiones asíncronas y comunicarse con el *backend*.

1.5. Estructura de la memoria

Para facilitar la lectura y comprensión de este documento, la memoria se ha estructurado en diez capítulos principales y dos manuales adicionales organizados

de la siguiente manera:

- **Capítulo 1: Introducción.** Define el punto de partida del proyecto, contextualizando el problema de la evaluación presencial frente al uso de herramientas digitales. Se exponen la motivación, los objetivos del proyecto y una visión general del *stack* tecnológico seleccionado.
- **Capítulo 2: Acta de constitución.** Formaliza el inicio del proyecto, detallando la descripción general del producto, los requerimientos de alto nivel, el marco legal y normativo (incluyendo cumplimiento de RGPD y licencias), los riesgos preliminares y los interesados o *stakeholders*.
- **Capítulo 3: Gestión del proyecto.** Describe la planificación organizativa inicial, incluyendo la metodología de trabajo, la línea base del alcance (a través de la EDT y su diccionario), la línea base del cronograma (con el diagrama de Gantt e hitos), la línea base de costes (estimación de presupuestos, Capex, Opex y TCO), así como los mecanismos para la gestión de riesgos y solicitudes de cambios.
- **Capítulo 4: Estado del arte y fundamentación.** Presenta un análisis comparativo de las soluciones y competidores existentes en el mercado de la gamificación educativa, identificando los factores diferenciales de *Quizzie*. Además, justifica detalladamente las decisiones tecnológicas del *stack* y las alternativas consideradas.
- **Capítulo 5: Análisis de requisitos.** Modela los requisitos del sistema empleando casos de uso, historias de usuario y reglas de negocio. Especifica tanto los requisitos funcionales como los no funcionales, y establece la matriz de trazabilidad que vincula los objetivos iniciales con las pruebas del sistema.
- **Capítulo 6: Diseño y arquitectura.** Detalla la arquitectura de *software* seleccionada (cliente-servidor estructurada en capas y basada en eventos), la modularización del código en el *frontend* y el *backend*, el modelo de datos relacional mediante diagramas de dominio, el diseño de la interfaz de usuario (*mockups* iniciales frente al acabado final), los mecanismos de seguridad (JWT y *tokens* de acceso) y la documentación de la API con OpenAPI/Swagger.
- **Capítulo 7: Metodología técnica (CI/CD).** Detalla los aspectos de ingeniería y DevOps integrados en el repositorio, abarcando las políticas de ramas y *commits* (Commitizen, *hooks* de *pre-commit*), el *pipeline* de integración y despliegue continuo (GitHub Actions), la gestión de perfiles de configuración y secretos, y el control de calidad estática mediante SonarQube.
- **Capítulo 8: Implementación y pruebas.** Recoge las decisiones del desarrollo real mostrando fragmentos de código significativos. Describe la estrategia de *testing* (pruebas unitarias, de integración y extremo a extremo), los umbrales de cobertura y el diseño de las pruebas de carga y rendimiento de la plataforma.
- **Capítulo 9: Resultados y evaluación.** Expone el análisis tras la finalización del desarrollo, comparando los datos de ejecución reales con la planificación inicial (desviaciones en alcance, tiempos y costes). Asimismo, presenta los

resultados de los casos de prueba ejecutados y una retrospectiva técnica enfocada en la **deuda técnica acumulada**.

- **Capítulo 10: Conclusiones.** Evalúa el **grado de consecución de los objetivos** generales e individuales planteados. Recoge las lecciones aprendidas durante el ciclo de vida del proyecto, una valoración personal sobre el proceso y las futuras líneas de evolución de la plataforma.

Adicionalmente, se incluyen dos manuales prácticos al final del documento como anexos independientes:

- **Manual de instalación:** Guía técnica detallada para el despliegue del sistema tanto en un entorno local de desarrollo como en entornos de producción en la nube.
- **Manual de usuario:** Guía visual e instructiva orientada a profesores y alumnos para el correcto uso y manejo de la plataforma.

2. Acta de constitución

El desarrollo de *Quizzie* requiere fijar un marco formal que defina sus reglas y compromisos iniciales antes de comenzar cualquier fase de diseño o desarrollo técnico. El propósito de este capítulo es establecer la línea base del proyecto, delimitando su alcance inicial, sus interesados y las restricciones bajo las que se ejecutará. Este documento define el propósito estratégico de la aplicación, garantizando desde el primer momento la viabilidad técnica, la gestión de riesgos y el estricto cumplimiento del marco legal establecido.

2.1. Información del proyecto

La formalización de *Quizzie* como Trabajo de Fin de Grado requiere establecer, en primer lugar, los datos de identificación administrativa y académica que muestran su autoría y tutorización.

A continuación, la Tabla 2.1 resume la información de referencia del proyecto:

Nombre del proyecto	Quizzie
Descripción	Plataforma multidispositivo para la realización y gestión dinámica de tests educativos
Tipo de proyecto	Trabajo de Fin de Grado (TFG)
Titulación	Grado en Ingeniería Informática - Ingeniería del Software
Autor	Francisco Gago Vázquez
Tutor	Jesús Moreno León
Departamento	Lenguajes y Sistemas Informáticos (LSI)

Tabla 2.1: Datos de identificación del proyecto.

2.2. Descripción y propósito

Quizzie nace como una plataforma multidispositivo orientada a transformar la dinámica de evaluación en el entorno educativo. El propósito fundamental de su desarrollo es ofrecer una herramienta interactiva que agilice la creación, gestión y ejecución de tests en tiempo real. Frente a soluciones comerciales existentes, la aplicación pone el foco en la inmediatez, la robustez técnica y control del flujo de

evaluación por parte del profesor gracias a la verificación de la presencialidad de los alumnos.

A diferencia de las metas de ingeniería y gestión académica planteadas en la sección 1.3, este apartado define los objetivos globales de la plataforma, los cuales se materializarán detalladamente en el catálogo de requisitos funcionales del sistema expuesto en el capítulo 5.

- **Control docente síncrono y monitorización:** Proveer al profesorado de herramientas dinámicas para guiar la sesión en tiempo real, permitiendo la gestión de salas y el control sobre los tiempos de respuesta.
- **Validación y presencialidad física:** Asegurar la legitimidad de las evaluaciones en el aula mediante mecanismos de verificación digital *in situ*, garantizando que el histórico de calificaciones refleje únicamente interacciones validadas.
- **Accesibilidad y agilidad de participación:** Optimizar la experiencia de usuario con flujos de acceso rápido sin registro para los estudiantes, facilitando una incorporación multidispositivo inmediata a la dinámica de la clase.
- **Automatización y análisis de rendimiento:** Incorporar capacidades de asistencia inteligente para agilizar el diseño de cuestionarios y estructurar analíticas de seguimiento que permitan evaluar la evolución del alumnado.
- **Garantía de privacidad y cumplimiento:** Blindar el tratamiento de los datos y el almacenamiento de resultados bajo el estricto cumplimiento del marco legal establecido.

2.3. Requerimientos de alto nivel

El éxito de la plataforma depende del cumplimiento de una serie de condiciones críticas, atributos de calidad y restricciones de diseño iniciales. Estas directrices sientan las bases del desarrollo y actúan como los criterios de aceptación esenciales para considerar el producto como terminado y listo para su lanzamiento.

Para validar el correcto estado del sistema, este debe cumplir estrictamente con la siguiente lista de verificación:

- **Disponibilidad y despliegue en la nube:** La plataforma completa debe estar desplegada en un entorno de producción real y accesible a través de internet para sus usuarios.
- **Compatibilidad multiplataforma:** La interfaz debe ser completamente *responsive*, garantizando que los alumnos interactúen de forma fluida desde dispositivos móviles y el profesor desde equipos de escritorio.
- **Simplicidad de uso:** El diseño de las pantallas debe priorizar la usabilidad, permitiendo el acceso a las salas de cuestionarios y la creación de test con el menor número de interacciones posible.

- **Seguridad en las comunicaciones:** El intercambio de datos en tiempo real debe realizarse bajo protocolos seguros (HTTPS y WSS), asegurando el cifrado de la información en tránsito.
- **Privacidad de los datos:** El almacenamiento del histórico de calificaciones y datos académicos debe cumplir estrictamente con el marco legal de protección de datos vigente.

2.4. Marco legal y normativo

El desarrollo y uso de *Quizzie* se rige por la normativa vigente en materia de protección de datos, seguridad de la información y propiedad intelectual. Al tratarse de una herramienta orientada al entorno educativo, el diseño del sistema ha priorizado el principio de minimización de datos para mitigar los riesgos asociados al tratamiento de información sensible.

A continuación, se detallan los pilares normativos y legales de la aplicación:

- **Reglamento General de Protección de Datos (RGPD):** La plataforma aplica el principio de privacidad desde el diseño. Para los estudiantes, el acceso a las salas se realiza mediante un código PIN y su *uvus* (*Usuario Virtual de la Universidad de Sevilla*), prescindiendo de un registro tradicional y limitando el almacenamiento de datos personales a este identificador institucional único. En el caso de los docentes, el registro obligatorio se limita a un correo electrónico y un alias, omitiendo identificadores directos como nombres, apellidos o DNI.
- **Seguridad y persistencia de calificaciones:** El histórico de resultados almacenado en el sistema solo vincula las calificaciones al *uvus* del alumno y cuenta con la previa verificación física del profesor. Esto garantiza la integridad del proceso de evaluación sin exponer la identidad real del estudiantado en la base de datos.
- **Propiedad intelectual y licenciamiento:** El código fuente y los entregables de este Trabajo de Fin de Grado se distribuyen bajo una licencia de código abierto MIT. Esta elección garantiza la total transparencia del desarrollo, permite la auditoría pública del sistema y fomenta su reutilización o extensión por parte de la comunidad académica, eximiendo al autor de cualquier responsabilidad derivada de su uso.

2.5. Riesgos iniciales de alto nivel

El desarrollo de una plataforma orientada a la evaluación en tiempo real conlleva una serie de riesgos técnicos y operativos. Identificarlos de forma temprana permite establecer estrategias de mitigación para garantizar el éxito del proyecto y el cumplimiento del cronograma.

A continuación, se presentan los riesgos iniciales detectados junto a sus respectivas medidas de contingencia:

Tipo	Nombre	Descripción	Mitigación
Técnico	Rendimiento síncrono	Saturación del servidor o retardos (<i>lag</i>) ante múltiples alumnos conectados a la vez en clase.	Diseñar un modelo de datos ligero para WebSockets y realizar pruebas de carga tempranas.
Técnico	Curva de aprendizaje	Retrasos por falta de experiencia previa con WebSockets o la integración de asistentes de IA.	Desarrollar prototipos aislados y estudiar acerca de nuevas tecnologías en las primeras semanas del proyecto.
Gestión	Alcance y tiempo	Los plazos estrictos de la universidad pueden peligrar ante la ambición del catálogo de funciones.	Adoptar metodología ágil y priorizar las tareas bajo un enfoque de Producto Mínimo Viable (MVP).
Operativo	Despliegue y red	Fallos de conectividad o restricciones en los puertos de la red wifi que impidan la conexión WebSocket.	Configurar el despliegue bajo protocolos seguros de producción y prever mecanismos alternativos de reconexión rápida.

Tabla 2.2: Riesgos iniciales y estrategias de mitigación.

2.6. Lista de interesados (*Stakeholders*)

En este apartado se identifica a los actores, grupos e instituciones que interactúan con la plataforma o se ven afectados por su desarrollo, definiendo sus roles y expectativas principales.

Interesado	Rol en el proyecto	Intereses / expectativas
Autor	Desarrollar e implementar el sistema completo.	Diseñar una arquitectura robusta, cumplir los plazos de entrega y superar con éxito la defensa del TFG.
Tutor	Supervisar, guiar y evaluar el desarrollo del proyecto.	Garantizar el rigor metodológico, la calidad técnica del software y validar el cumplimiento de los objetivos.
Departamento LSI	Entidad académica de la Universidad responsable de la calificación del TFG.	Asegurar el cumplimiento de la normativa de los Trabajos de Fin de Grado y el correcto proceso de evaluación.
Cuerpo docente	Futuros usuarios (Profesores).	Disponer de una herramienta intuitiva que facilite la evaluación de los alumnos y asegure su presencialidad.
Alumnado	Futuros usuarios (Estudiantes).	Acceder de forma ágil y multidispositivo sin registros complejos para participar en las evaluaciones dinámicas.

Tabla 2.3: Lista de interesados y roles en el proyecto.

3. Gestión del proyecto

En la ingeniería de software, el éxito de un proyecto no depende únicamente de la calidad de su código, sino de una rigurosa gobernanza que garantice la viabilidad técnica, temporal y económica del proyecto. Para ello, se adopta un enfoque predictivo en la definición de las líneas base, complementado con prácticas ágiles para la gestión y ejecución del cambio.

A lo largo de las siguientes secciones se detalla la metodología que se seguirá a lo largo del desarrollo, se establecen formalmente las línea base del alcance, cronograma y de costes, y se presenta el plan de gestión de riesgos y cambios destinado a mitigar las desviaciones del proyecto.

3.1. Metodología del trabajo

El desarrollo de *Quizzie* se organiza gracias a un flujo de trabajo ágil centralizado en GitHub. Para el día a día y el seguimiento del proyecto, se utilizan las siguientes herramientas:

- **GitHub:** Centraliza el almacenamiento del código fuente y la gestión de las tareas mediante las siguientes herramientas:
 - **Issues y Milestones:** Cada funcionalidad o corrección se registra como una *Issue* (tarea). Estas se agrupan en *Milestones* (hitos temporales) para asociar el avance del código con los plazos asignados en el cronograma.
 - **Ramas permanentes:** Se mantiene una separación estricta entre la rama *main* (exclusiva para versiones definitivas y estables) y la rama *develop* (entorno de integración donde se vuelcan las novedades).
 - **Ramas de tareas:** Para cada *issue* se abre una rama específica desde *develop* (ej. *feature/websocket-timer*). Una vez completada y probada la tarea de forma aislada, esta rama se fusiona (*merge*) de vuelta en *develop*.
 - **Releases:** Al completar un *Milestone* y consolidar los cambios en *main*, se genera una *Release* con su etiqueta de versión.
- **Clockify:** Se utiliza exclusivamente para cronometrar el tiempo real dedicado a cada tarea e hito, permitiendo controlar el esfuerzo invertido.
- **Outlook:** Canal formal para comunicaciones con el tutor y envío de entregables o documentación.

El avance del proyecto se valida de forma continua mediante tutorías. La dinámica de estas reuniones consiste en presentar y revisar el software funcionando o el texto redactado hasta esa fecha para recibir el visto bueno del

tutor y, acto seguido, definir los siguientes pasos y prioridades a realizar para la próxima tutoría.

3.2. Línea base del alcance

La línea base del alcance define con precisión los límites del proyecto, asegurando que todos los esfuerzos se concentren exclusivamente en los entregables necesarios para cumplir con los objetivos académicos y técnicos. A continuación, se detallan los supuestos, restricciones y exclusiones que condicionan esta planificación.

- **Supuestos:** Se asume la disponibilidad continua de las infraestructuras en la nube (proveedores de alojamiento o bases de datos) durante las fases de desarrollo y despliegue, así como el acceso ilimitado a la documentación de las API y tecnologías que componen el *stack* tecnológico.
- **Restricciones:** El proyecto está sujeto a una restricción temporal vinculada a los plazos institucionales de entrega y defensa del Trabajo Fin de Grado. Asimismo, el desarrollo debe ceñirse al presupuesto simulado y a la capacidad operativa de un único desarrollador.
- **Exclusiones:** Queda fuera del alcance del proyecto el despliegue comercial en producción y el soporte para navegadores web obsoletos o sin compatibilidad con WebSockets modernos.

3.2.1. Estructura del Desglose del Trabajo (EDT)

Para organizar y estructurar el alcance total del proyecto, se ha elaborado la siguiente Estructura de Desglose del Trabajo (EDT). Esta representación jerárquica descompone el proyecto en paquetes de trabajo, divididos a su vez en entregables específicos que se detallarán en el diccionario de la EDT ([3.2.2](#)).

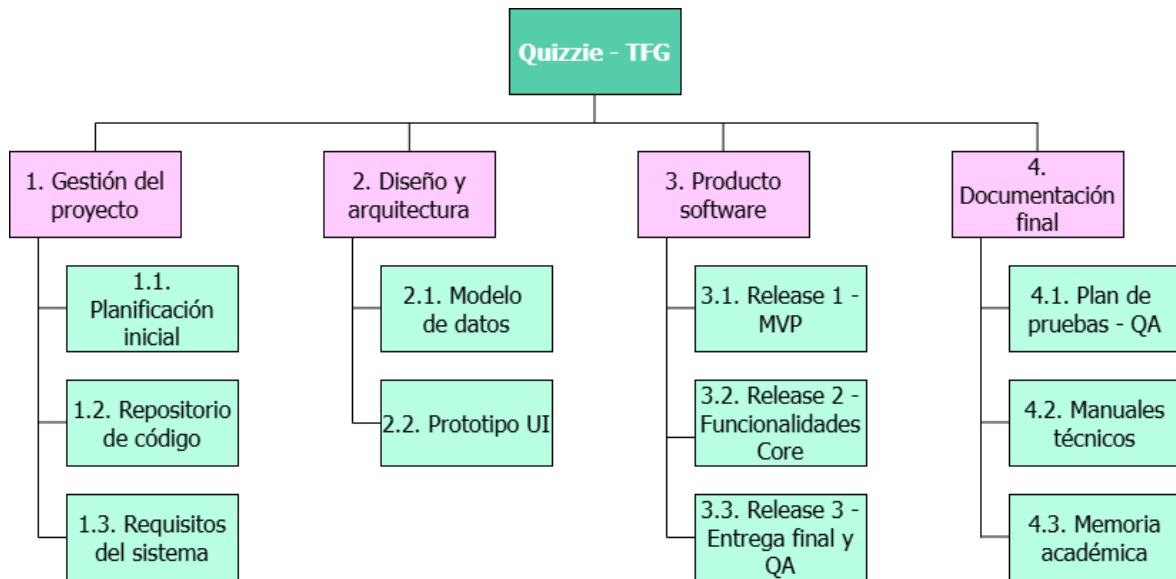


Figura 3.1: Estructura de Desglose del Trabajo (EDT).

3.2.2. Diccionario de la EDT

El diccionario de la EDT define formalmente el alcance de cada entregable y desglosa las actividades necesarias para su consecución, especificando los perfiles profesionales asignados y la estimación temporal.

Paquete 1.0: Gestión del proyecto

ID Entregable	1.1
Nombre	Planificación inicial
Descripción	Acta de constitución del proyecto y cronograma base estimado para fijar los hitos académicos.
Paquete	1. Gestión del proyecto

Tabla 3.1: Datos del entregable 1.1.

Actividad	Recurso	Horas
Redacción del acta de constitución	Project Manager	4h
Definición de hitos y estimación del cronograma inicial	Project Manager	4h

Tabla 3.2: Actividades del entregable 1.1.

ID Entregable	1.2
Nombre	Repositorio de código
Descripción	Configuración de GitHub con reglas de protección de ramas, flujos de trabajo basados en fusiones (<i>merges</i>) y plantillas estandarizadas para <i>issues</i> .
Paquete	1. Gestión del proyecto

Tabla 3.3: Datos del entregable 1.2.

Actividad	Recurso	Horas
Inicialización del repositorio en GitHub y estructura base	Técnico	2h
Configuración de reglas en ramas y automatización base	Project Manager	5h
Diseño y despliegue de <i>templates</i> para <i>issues</i>	Project Manager	3h

Tabla 3.4: Actividades del entregable 1.2.

ID Entregable	1.3
Nombre	Requisitos del sistema
Descripción	Catálogo de requisitos funcionales y no funcionales, junto al modelado de casos de uso y flujos <i>core</i> de la aplicación.
Paquete	1. Gestión del proyecto

Tabla 3.5: Datos del entregable 1.3.

Actividad	Recurso	Horas
Elicitación y redacción de requisitos funcionales y no funcionales	Analista	3h
Modelado de diagramas de casos de uso y flujos esenciales	Analista	4h

Tabla 3.6: Actividades del entregable 1.3.

Paquete 2.0: Diseño y Arquitectura

ID Entregable	2.1
Nombre	Modelo de datos
Descripción	Diseño conceptual, lógico y físico de la base de datos relacional, materializado en el diagrama de entidades del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.7: Datos del entregable 2.1.

Actividad	Recurso	Horas
Diseño del modelo conceptual y lógico de datos	Analista	4h
Modelado del diagrama de entidades relacional	Analista	4h
Generación del <i>script</i> de inicialización de tablas y poblado de datos	Programador Backend	2h

Tabla 3.8: Actividades del entregable 2.1.

ID Entregable	2.2
Nombre	Prototipo UI
Descripción	Diseños visuales y plantillas para las pantallas del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.9: Datos del entregable 2.2.

Actividad	Recurso	Horas
Elección de estilos, paleta de colores y componentes básicos de interfaz	Diseñador UI	2h
Diseño de prototipos de las pantallas de gestión del profesor	Diseñador UI	5h
Diseño de la interfaz de respuesta para pantallas de alumnos	Diseñador UI	5h

Tabla 3.10: Actividades del entregable 2.2.

Paquete 3.0: Producto software

ID Entregable	3.1
Nombre	Release 1 (MVP)
Descripción	Habilitación y despliegue del flujo completo base del sistema, permitiendo la creación de cuestionarios por parte del profesor y la realización de los mismos por los alumnos.
Paquete	3. Producto software

Tabla 3.11: Datos del entregable 3.1.

Actividad	Recurso	Horas
Diseño y desarrollo de la persistencia base para cuestionarios y preguntas	Programador Backend	25h
Implementación de la interfaz de creación de tests y panel de respuestas básico	Programador Frontend	35h
Lógica de control y validación para la ejecución asíncrona del flujo de cuestionarios	Programador Backend	20h

Tabla 3.12: Actividades del entregable 3.1.

ID Entregable	3.2
Nombre	Release 2 (Funcionalidades Core)
Descripción	Implementación de la capa de comunicación en tiempo real mediante WebSockets y el motor de validación de datos del sistema.
Paquete	3. Producto software

Tabla 3.13: Datos del entregable 3.2.

Actividad	Recurso	Horas
Desarrollo de la infraestructura síncrona del servidor y gestión de canales WebSocket	Programador Backend	20h
Implementación en frontend de la conexión reactiva a las salas en tiempo real	Programador Frontend	20h
Programación del motor de validación y control de integridad de datos	Programador Backend	12h

Tabla 3.14: Actividades del entregable 3.2.

ID Entregable	3.3
Nombre	Release 3 (Entrega final y QA)
Descripción	Integración de funcionalidades avanzadas de gestión, pulido general de la interfaz y despliegue del sistema.
Paquete	3. Producto software

Tabla 3.15: Datos del entregable 3.3.

Actividad	Recurso	Horas
Implementación del módulo de autenticación de usuarios y panel de gestión avanzada de salas	Programador Backend	5h
Desarrollo de las vistas de historial de salas pasadas y sistema de exportación de datos	Programador Frontend	5h
Puesta a punto, empaquetado, corrección de errores de la interfaz y despliegue definitivo	Técnico	3h

Tabla 3.16: Actividades del entregable 3.3.

Paquete 4.0: Documentación final

ID Entregable	4.1
Nombre	Plan de pruebas (QA)
Descripción	Diseño y ejecución de baterías de test de integración y <i>scripts</i> específicos de pruebas de carga.
Paquete	4. Documentación final

Tabla 3.17: Datos del entregable 4.1.

Actividad	Recurso	Horas
Desarrollo y ejecución de test de integración de servicios	Tester	7h
Programación y lanzamiento de <i>scripts</i> de carga	Tester	8h

Tabla 3.18: Actividades del entregable 4.1.

ID Entregable	4.2
Nombre	Manuales técnicos
Descripción	Redacción del manual de usuario y del manual de despliegue.
Paquete	4. Documentación final

Tabla 3.19: Datos del entregable 4.2.

Actividad	Recurso	Horas
Elaboración del manual funcional como guía para usuarios	Analista	2h
Redacción del manual de despliegue técnico de infraestructura	Técnico	1h

Tabla 3.20: Actividades del entregable 4.2.

ID Entregable	4.3
Nombre	Memoria académica
Descripción	Redacción técnica, maquetación y consolidación del documento final del TFG junto con el diseño de las diapositivas para la defensa.
Paquete	4. Documentación final

Tabla 3.21: Datos del entregable 4.3.

Actividad	Recurso	Horas
Redacción y estructuración técnica de la memoria en LaTeX	Analista	70h
Diseño, desarrollo y maquetación de las diapositivas de la defensa	Diseñador UI	10h
Ensayos de oratoria y preparación de la defensa pública	Project Manager	10h

Tabla 3.22: Actividades del entregable 4.3.

3.3. Línea Base del Cronograma

3.3.1. Lista de actividades

3.3.2. Lista de hitos

3.3.3. Diagrama de Gantt

Cronograma con actividades e hitos

Mi duda sería de qué forma hacer los cronogramas y diagramas de Gantt, si con alguna herramienta específica o si con tablas o gráficos hechos a mano, y también qué nivel de detalle incluir en ellos

3.4. Línea Base de Costes

Aquí me surgen las mismas dudas que en el cronograma, quizás podría apoyarme en un Excel para los costes. También dudo en si tengo que poner costes de herramientas como Github o Gemini, y los costes de despliegue

3.4.1. Análisis de costes

Incluyendo Capex y Opex

3.4.2. Costes de infraestructura según el número de usuarios

3.4.3. Coste total de propiedad (TCO)

3.4.4. Plan de gestión de cambios y riesgos

Cómo se gestionarán los cambios en el proyecto y los riesgos identificados

Quiero dividir los riesgos en temporales, de alcance y técnicos (económicos no le veo mucho sentido pero también podría ser), y quizás definir roles encargados de solicitar cambios, aprobarlos y demás

4. Estado del arte y fundamentación

Este es quizás, el capítulo que menos claro tengo cómo enfocarlo

4.1. Análisis del mercado

Análisis de aplicaciones similares, competidores y soluciones existentes

4.2. Factores diferenciadores

Elementos que hacen que el proyecto sea único o innovador en comparación con las soluciones existentes

4.3. Fundamentación tecnológica

Tecnologías de Backend, Frontend, BBDD, infraestructura y por qué se han elegido Herramientas de soporte al desarrollo (IDE, control de versiones, etc.)

Aquí se ampliaría lo dicho en la introducción sobre el stack tecnológico, explicando por qué se han elegido y adentrándonos en todas las herramientas utilizadas

Hay que ver si poner las siguientes subsecciones o dividir por cada una de las tecnologías seleccionadas y hacer una tabla

4.3.1. Limitaciones tecnológicas

4.3.2. Alternativas consideradas

4.3.3. Justificación de la decisión

4.4. Conclusiones del estudio previo

Decir por qué veo necesario el proyecto, qué aporta y cómo se va a llevar a cabo

5. Análisis de requisitos

Aquí se describirán los objetivos del sistema

5.1. Actores del sistema

Alumnos, profesores (si procede, también administrador)

5.2. Historias de usuario

Como [actor], quiero [acción] para [beneficio]”.

Valorar si incluirlos todos en una diagrama estilo draw.io

5.3. Requisitos de información

5.4. Reglas de negocio y restricciones

5.5. Requisitos funcionales

5.5.1. Criterios de aceptación

Se puede poner dentro de los RF como subsección o fuera como sección, enlazar con los casos de prueba y validación (cap 8.3)

5.6. Casos de uso

Descripción y después diagrama parecido al del de historias de usuario, pero un diagrama por CU

5.7. Requisitos no funcionales

Requisitos de rendimiento, seguridad, usabilidad, etc.

5.8. Trazabilidad de requisitos

Matriz de trazabilidad : Objetivos → Requisitos → Diseño → Pruebas → Resultados

6. Diseño y arquitectura

6.1. Patrón arquitectónico elegido

Arquitectura Cliente-Servidor, arquitectura en capas y arquitectura basada en eventos

Hablar de los patrones arquitectónicos elegidos, explicando por qué se han seleccionado y cómo se aplican en el diseño de la aplicación, destacando la separación de responsabilidades y la escalabilidad que ofrecen.

6.2. Estructura modular del código

Organización del proyecto en módulos independientes tanto para el Frontend (React) como para el Backend (FastAPI), facilitando la escalabilidad y el mantenimiento

Hablar de la organización del código en módulos independientes, explicando cómo se han estructurado y por qué es importante para la escalabilidad y el mantenimiento del proyecto.

6.3. Modelo de datos

Diagrama de dominio

Incluir el código mermaid y el diagrama

6.4. Diseño de interfaces

Mockups y resultado final

Mostrar los mockups iniciales y compararlos con el resultado final, explicando las decisiones de diseño tomadas y cómo se han implementado en la interfaz de usuario.

6.5. Seguridad

Autenticación JWT, gestión de roles y tokens de verificación para participantes

6.6. Documentación de la API

Descripción de los endpoints principales de la API, detallando las peticiones, respuestas y la integración con el estándar OpenAPI

Aquí exprimiría el Swagger

7. Metodología técnica (CI/CD)

Mi idea es haber explicado todas las herramientas en el capítulo 4, pero aquí se podría entrar más en detalle sobre cómo se han configurado y utilizado en lo que respecta a ci/cd, gestión de ramas, commits, etc. También se podrían incluir políticas de código, como pre-commits, hooks, plantillas de issues, etc. que no se hayan mencionado en capítulos anteriores.

7.1. Políticas del repositorio

Ramas, commits y versiones. Se puede mencionar los pre-commits, hooks, Github, Commitizen, Convenciones de commits, plantillas de issues, ... y su importancia en el desarrollo para trabajar de manera más organizada

7.2. Integración y Despliegue Continuo (CI/CD)

Descripción del ciclo ci/cd

7.2.1. Configuración del Pipeline y automatización

Implementación de flujos de trabajo en GitHub Actions y qué herramientas incluyen

7.3. Perfiles de configuración y entornos

Variables de entorno, secretos y configuración específica para los entornos de desarrollo, pruebas y producción.

7.4. Métricas de calidad y análisis estático de código

SonarCloud para el análisis estático de código, deuda técnica y cobertura de pruebas. (Para ello también habría que ver hasta qué punto se ha hablado de Sonar en capítulos anteriores)

Aquí quiero hacer especial énfasis para explicar la importancia de estas métricas, cómo se han configurado y qué resultados han dado, mostrando gráficas o ejemplos concretos de cómo han ayudado a mejorar la calidad del código y a

identificar áreas de mejora. También es importante recalcar que su uso es vital para evitar deuda técnica, y en el capítulo 9 de resultados diré que me arrepiento de haberlo implementado tan tarde y que me ha hecho tener que aplazar un par de issues que tenias planteadas para la entrega del tfg. (también podría ser en el capítulo de conclusiones)

8. Implementación y pruebas

8.1. Componentes clave

Fragmentos de código fundamentales

8.2. Estrategia de testing

Unitarias, integración, E2E, etc

Me gustaría mencionar también su importancia y los objetivos de cobertura que tengo, quizás lo mejor sea en este capítulo mencionar los umbrales objetivos y en el capítulo 9 de resultados hablar de su cumplimiento

8.3. Casos de prueba y validación

Descripción de los casos de prueba utilizados para validar los criterios de aceptación y alcanzar la cobertura objetivo

8.4. Pruebas de carga y rendimiento

Descripción de las pruebas de carga y rendimiento realizadas

Importante porque los cuestionarios son en directo y con muchas personas simultáneamente

9. Resultados y evaluación

9.1. Grado de cumplimiento de la Línea Base del Alcance

Evaluación del cumplimiento de los objetivos y requisitos establecidos en la Línea Base del Alcance.

9.2. Tiempo real frente a Línea Base del Cronograma

Comparación del tiempo real dedicado a cada fase del proyecto con el tiempo estimado inicialmente

9.3. Gastos incurridos frente a Línea Base de Costes

Diferencia entre los gastos previstos y los reales

9.4. Desviaciones y gestión de cambios realizados

Cómo se gestionaron las desviaciones y qué modificaciones de la planificación provocaron

9.5. Resultados de las pruebas y validación

Resultados obtenidos en las pruebas unitarias, de integración, E2E, carga y rendimiento, y su comparación con los objetivos establecidos

9.6. Retrospectiva técnica

Reflexión sobre la implementación de las prácticas de CI/CD, testing y métricas de calidad, y decir si he cumplido los objetivos como las métricas de cobertura o de issues de sonar

10. Conclusiones

10.1. Grado de cumplimiento de los objetivos

Evaluación sobre los objetivos planteados en el acta de constitución.

10.2. Lecciones aprendidas y valoración personal

Reflexión sobre el trabajo realizado.

10.3. Trabajos futuros y líneas de evolución

Conclusiones sobre el trabajo realizado y líneas de evolución futuras.

Manual de usuario

Manual de usuario de quizzie

Manual de instalación

Guía de instalación

Bibliografía

- [1] Agustín Borrego, Daniel Ayala, Inma Hernández, Carlos R. Rivero, and David Ruiz. Generating rules to filter candidate triples for their correctness checking by knowledge graph completion techniques. In *K-CAP*, pages 115–122, 2019. doi: 10.1145/3360901.3364418.